

Name: Shreyas B Patil SRN: PES2UG24CS488 LAB:4 SECTION: H

LLM: chatgpt link : <https://chatgpt.com/share/6abcd4d1-4c68-83ee-b176-b1c655b17305>

Task 1:

 PES2UG24CS488_LAB4-main.zip
Zip Archive

 Free offer 

i am gamer/coder whose objective is to fix the brick baker game. i have sent the zip file containing what the codes as well as readme. the readme has the tasks mentioned . pls help me complete the task

Worked for 1m 11s >

Done. I went through the README and the source code and implemented all 4 required tasks.

What was fixed

- 1. Collision detection**
 - Detects whether the ball hits the **top, bottom, left, or right** of a brick/paddle.
 - Horizontal hits now reverse `vx`.
 - Vertical hits reverse `vy`.
 - The ball is repositioned after collisions to reduce clipping through bricks.
- 2. Game Over / Win screen**
 - No more console-only messages.
 - Displays:
 - `YOU WIN!` when all bricks are destroyed.
 - `GAME OVER` when all lives are lost.
 - Final score.

- 3. Replay + difficulty**
 - After the game ends:
 - `1` → Easy: slower ball + larger paddle
 - `2` → Medium
 - `3` → Hard: faster ball + smaller paddle
 - `Q` / `Esc` → Exit
- 4. Sound effects**
 - Brick hit
 - Paddle hit
 - Wall hit
 - Win
 - Lose
 - Sounds are generated in memory, so no additional `.wav` files are required.

Completed project

Download the completed Brick Breaker project ZIP

One thing to note: I verified that the modified Python files pass a syntax/compile check. I couldn't launch the graphical game in my environment because `pygame` isn't installed here, but your `requirements.txt` already contains `pygame`, so on your machine you just need:

as a gamer i would like to go task by task manually updating it and also task by task

Absolutely — that is actually a better way to do this for a lab. We'll **not use my completed ZIP**. We'll work from your original project and make **one task at a time**, with you manually editing the files and testing after each task.

We'll follow this order:

1. **Task 1 — Fix collision detection**
2. Test Task 1 and make sure the game still works
3. **Task 2 — Game Over / Win screen**
4. Test Task 2
5. **Task 3 — Replay + difficulty selection**
6. Test Task 3
7. **Task 4 — Sound effects**
8. Final testing and README/report requirements

Step 1. Open `game/game_engine.py`

 Free c

Find this section:

```
</> Python
```

```
def update(self):
    if self.game_over:
        return

    self.ball.move()
```

  Run

We are going to make a small change **right here**.

Replace:

```
</> Python
```

```
self.ball.move()
```

  Run

with:

```
</> Python
```

```
previous_rect = self.ball.rect()
self.ball.move()
current_rect = self.ball.rect()
```

  Run

Step 2 — Fix the paddle collision

 F

Find this existing code:

```
</> Python
```

```
if self.ball.rect().colliderect(self.paddle.rect()):
    # NOTE: always flips the ball's vertical velocity on a
    # paddle collision, regardless of which side of the paddle
    # was actually hit. See Task 1 in the README.
    self.ball.vy *= -1
```

  Run

For now, **replace the entire block with**:

```
</> Python
```

```
if current_rect.colliderect(self.paddle.rect()):
    paddle_rect = self.paddle.rect()

    if previous_rect.bottom <= paddle_rect.top and current_rect.bottom >= paddle_rect.top:
        self.ball.vy = -abs(self.ball.vy)

    elif previous_rect.top >= paddle_rect.bottom and current_rect.top <= paddle_rect.bottom:
        self.ball.vy = abs(self.ball.vy)

    elif previous_rect.right <= paddle_rect.left and current_rect.right >= paddle_rect.left:
        self.ball.vx = -abs(self.ball.vx)

    elif previous_rect.left >= paddle_rect.right and current_rect.left <= paddle_rect.right:
        self.ball.vx = abs(self.ball.vx)
```

  Run



Step 3 — Fix the brick collision



Now find this block:

```
</> Python
```

```
for brick in self.bricks:
    if brick.alive and self.ball.rect().colliderect(brick.rect()):
        brick.alive = False
        self.score += 1
        # NOTE: same unconditional vertical-velocity flip as
        # the paddle collision above - a brick hit from the
        # left or right should redirect the ball sideways
        # (flip vx), but this always flips vy instead. See
        # Task 1 in the README.
        self.ball.vy *= -1
        break
```

  Run

Replace it with:



`</>` Python



Run

```
for brick in self.bricks:
    if brick.alive and current_rect.colliderect(brick.rect()):
        brick_rect = brick.rect()

        brick.alive = False
        self.score += 1

    if previous_rect.bottom <= brick_rect.top and current_rect.bottom >= brick_rect.top:
        self.ball.vy = -abs(self.ball.vy)

    elif previous_rect.top >= brick_rect.bottom and current_rect.top <= brick_rect.bottom:
        self.ball.vy = abs(self.ball.vy)

    elif previous_rect.right <= brick_rect.left and current_rect.right >= brick_rect.left:
        self.ball.vx = -abs(self.ball.vx)

    elif previous_rect.left >= brick_rect.right and current_rect.left <= brick_rect.right:
        self.ball.vx = abs(self.ball.vx)

break
```

Task 2:

yes it has been completed we can move to task 2

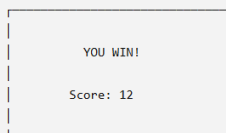
Great. Task 1 is complete. ✓

Let's move to Task 2 — Game Over / Win Screen.

The goal here is simple: instead of the game only printing something in the terminal when you win or lose, we want an actual screen inside the game.

What we want

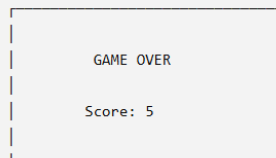
When all bricks are destroyed:



YOU WIN!
Score: 12



And when the player loses all lives:



GAME OVER
Score: 5



At the very bottom of your `render()` function, you currently have:

 Python



 Run

```
if self.game_over and not getattr(self, "_game_over_logged", False):
    # NOTE: no proper end screen yet - see Task 2 in the README.
    if self.result == "win":
        print("You win! Final score:", self.score)
    else:
        print("Game over! Final score:", self.score)
    self._game_over_logged = True
```

This is what we're going to replace.

Delete that entire block.

Replace it with:

 Python



 Run

```
if self.game_over:
    if self.result == "win":
        message = "YOU WIN!"
    else:
        message = "GAME OVER"

    result_text = self.font.render(message, True, WHITE)
    score_text = self.font.render(
        f"Final Score: {self.score}", True, WHITE
    )

    screen.blit(
        result_text,
        (
            self.width // 2 - result_text.get_width() // 2,
            self.height // 2 - 40,
        ),
    )

    screen.blit(
        score_text,
        (
            self.width // 2 - score_text.get_width() // 2,
            self.height // 2 + 10,
        ),
    )
```



Task 3:

task 2 has been done we have finished the object of printing game over and win. we move to task 3 the objective is the replay option right

Exactly.  Task 2 is complete.

For Task 3, the main objective is the **replay option**, but according to the task requirements, we should implement the **difficulty selection + replay flow** together.

The desired behavior is:

```
GAME OVER
Final Score: 12
```

```
1 - Easy
2 - Medium
3 - Hard
Q - Quit
```

The player chooses a difficulty, and the game **restarts**.

For example:

- **Easy** → slower ball / easier paddle
- **Medium** → normal settings
- **Hard** → faster ball / harder paddle

We will do this **manually and incrementally**, just like tasks 1 and 2.

Task 3 — Step 1: Add a reset method

Open:

game/game_engine.py

We already have this method:

Python

```
def _reset_ball(self):
    self.ball.x, self.ball.y = self.width // 2, self.height - 50
    self.ball.vx, self.ball.vy = 4, -4
```

Immediately **after** this method, add a new method:

Python

```
def reset_game(self):
    self.paddle = Paddle(self.width // 2 - 50, self.height - 30, 100, 14)
    self.ball = Ball(self.width // 2, self.height - 50, radius=8)
    self.ball.vx, self.ball.vy = 4, -4

    self.bricks = self._build_bricks(self.rows, self.cols)

    self.lives = 3
    self.score = 0
    self.game_over = False
    self.result = None
```

Task 3 — Step 2: Add difficulty selection

We'll make the three choices:

Key	Difficulty	Ball speed	Paddle
1	Easy	3	Larger
2	Medium	4	Normal
3	Hard	6	Smaller

The exact values aren't important; what matters is that the difficulty visibly changes the gameplay.

1. Modify `reset_game()`

Currently you added:

 Python



 Run

```
def reset_game(self):
    self.paddle = Paddle(self.width // 2 - 50, self.height - 30, 100, 14)
    self.ball = Ball(self.width // 2, self.height - 50, radius=8)
    self.ball.vx, self.ball.vy = 4, -4

    self.bricks = self._build_bricks(self.rows, self.cols)

    self.lives = 3
    self.score = 0
    self.game_over = False
    self.result = None
```

Change the first part so that it accepts a difficulty:



 Python



 Run

```
def reset_game(self, difficulty="medium"):
    if difficulty == "easy":
        paddle_width = 120
        ball_speed = 3

    elif difficulty == "hard":
        paddle_width = 80
        ball_speed = 6

    else:
        paddle_width = 100
        ball_speed = 4

    self.paddle = Paddle(
        self.width // 2 - paddle_width // 2,
        self.height - 30,
        paddle_width,
        14
    )

    self.ball = Ball(
        self.width // 2,
        self.height - 50,
        radius=8
    )
```

```
self.ball.vx = ball_speed
self.ball.vy = -ball_speed

self.bricks = self._build_bricks(self.rows, self.cols)

self.lives = 3
self.score = 0
self.game_over = False
self.result = None
```

Step 3: Handle the keyboard selection



Now open `game_engine.py` and find:

```
Python
def handle_event(self, event):
    # This game only needs continuously-held-key input for the
    # paddle, handled in handle_input each frame.
    pass
```

Replace the entire method with:

```
Python
def handle_event(self, event):
    if not self.game_over:
        return

    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_1:
            self.reset_game("easy")

        elif event.key == pygame.K_2:
            self.reset_game("medium")

        elif event.key == pygame.K_3:
            self.reset_game("hard")
```

Task 3 — Step 4: Display the replay menu

We'll modify only the `render()` function in `game_engine.py`.

You currently have this:

```
Python
if self.game_over:
    if self.result == "win":
        message = "YOU WIN!"
    else:
        message = "GAME OVER"

    result_text = self.font.render(message, True, WHITE)

    score_text = self.font.render(
        f"Final Score: {self.score}", True, WHITE
    )

    screen.blit(
        result_text,
        (
            self.width // 2 - result_text.get_width() // 2,
            self.height // 2 - 40,
        ),
    )
```

```
screen.blit(
    score_text,
    (
        self.width // 2 - score_text.get_width() // 2,
        self.height // 2 + 10,
    ),
)
```

Add this immediately after the `screen.blit(score_text, ...)` block:

```
Python

replay_text = self.font.render(
    "1 - Easy  2 - Medium  3 - Hard",
    True,
    WHITE
)

quit_text = self.font.render(
    "Q - Quit",
    True,
    WHITE
)

screen.blit(
    replay_text,
    (
        self.width // 2 - replay_text.get_width() // 2,
        self.height // 2 + 60,
    ),
)

screen.blit(
    quit_text,
    (
        self.width // 2 - quit_text.get_width() // 2,
        self.height // 2 + 100,
    ),
)
```

Step 5 — Add the `Q` option

We already added `1`, `2`, and `3` to `handle_event()`.

Currently you have:

```
Python

def handle_event(self, event):
    if not self.game_over:
        return

    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_1:
            self.reset_game("easy")

        elif event.key == pygame.K_2:
            self.reset_game("medium")

        elif event.key == pygame.K_3:
            self.reset_game("hard")
```

Add this at the end:

```
Python

elif event.key == pygame.K_q:
    pygame.event.post(pygame.event.Event(pygame.QUIT))
```

Task 4:

done it worked  Free of

Excellent. 🎮 Task 3 is officially complete. ✓

We now have:

- 1 → Easy
- 2 → Medium
- 3 → Hard
- Q → Quit
- Game resets correctly
- Difficulty actually changes ball/paddle behavior
- Options are displayed on the end screen

Task 4 — Sound Effects 🔊

This is the final task.

We'll add sounds for the important game events:

1. Ball hits wall
2. Ball hits paddle
3. Ball hits brick
4. Player wins



Step 1 — Initialize the audio mixer

Open `main.py`.

Immediately after:

```
</> Python
pygame.init()
```



Run

add:

```
</> Python
pygame.mixer.init()
```



Run

So the beginning should look like:

```
</> Python
import pygame
from game.game_engine import GameEngine

pygame.init()
pygame.mixer.init()

WIDTH, HEIGHT = 640, 560
```



Run



Step 2 — Create a sound folder

Inside your project, create a folder:

```
BrickBreaker/
├── main.py
├── game/
│   ├── game_engine.py
│   ├── paddle.py
│   ├── ball.py
│   └── brick.py
└── sounds/
```

For now, **just create the `sounds` folder**.

We need a few sound effects eventually, such as:

- `paddle.wav` — ball hits paddle
- `brick.wav` — ball hits brick
- `lose.wav` — player loses a life
- `win.wav` — player wins

You can use any short `.wav` sound effects you have available.

Don't modify the Python files yet. Create the folder and tell me when it's done.



Step 3 — Add the sound files

For the next step, let's keep it simple and use **two sounds first**:

```
sounds/
├── paddle.wav
└── brick.wav
```

You can download or use any short `.wav` sound effects. They don't need to be special—the important thing is that they are valid WAV files.

Once you have both files inside `sounds/`, **don't change the Python code yet**.

Tell me when they're there, and we'll load them into `GameEngine` next.



Step 4 — Put them in the `sounds` folder

Rename them to:

```
paddle.mp3
brick.mp3
```

and place them here:

```
sounds/
├── paddle.mp3
└── brick.mp3
```

Then tell me once you've done that.

Step 5 — Load the sounds

Open:

game/game_engine.py



Go to the `__init__()` method of `GameEngine`.

At the end of `__init__()`, add:

Python



Run

```
self.paddle_sound = pygame.mixer.Sound("sounds/paddle.mp3")
self.brick_sound = pygame.mixer.Sound("sounds/brick.mp3")
```

So you'll have something like:

Python



Run

```
def __init__(self, width, height):
    # existing code...

    self.paddle_sound = pygame.mixer.Sound("sounds/paddle.mp3")
    self.brick_sound = pygame.mixer.Sound("sounds/brick.mp3")
```

Step 6 — Play the paddle sound

Now we'll connect **only the paddle sound** to the paddle collision.

In `game/game_engine.py`, find the code we added earlier for paddle collision:

Python



Run

```
if current_rect.colliderect(self.paddle.rect()):
    paddle_rect = self.paddle.rect()

    if previous_rect.bottom <= paddle_rect.top and current_rect.bottom >= paddle_rect.top:
        self.ball.vy = -abs(self.ball.vy)
```

We want the sound to play **when the collision is detected**, regardless of whether the collision is from the top, bottom, left, or right.

So add this line immediately after:

Python



Run

```
if current_rect.colliderect(self.paddle.rect()):
```

Add:

Python



Run

```
self.paddle_sound.play()
```



Step 7 — Add the brick sound

Now we'll make **just one change** for brick collisions.

In `game/game_engine.py`, find the section where you detect a brick collision. It should look roughly like:

Python

```
if current_rect.colliderect(brick.rect()):
    brick.alive = False
    self.score += 1
```

Add:

Python

```
self.brick_sound.play()
```

So it becomes:

Python

```
if current_rect.colliderect(brick.rect()):
    self.brick_sound.play()
    brick.alive = False
    self.score += 1
```

Step 8 — Add a lose sound

You currently only have two sound files. Do you have another short sound that would work for **losing a life / Game Over**?

Something like:

- a short descending tone
- buzzer
- fail sound
- negative/error beep

If you have one, put it in `sounds/` as:

```
sounds/
├─ paddle.mp3
├─ brick.mp3
└─ lose.mp3
```

Don't change the Python code yet.

Step 9 — Load `lose.mp3`

In `game/game_engine.py`, inside `GameEngine.__init__()`, you currently have:

Python

```
self.paddle_sound = pygame.mixer.Sound("sounds/paddle.mp3")
self.brick_sound = pygame.mixer.Sound("sounds/brick.mp3")
```

Add one line below them:

Python

```
self.lose_sound = pygame.mixer.Sound("sounds/lose.mp3")
```

So:

Python

```
self.paddle_sound = pygame.mixer.Sound("sounds/paddle.mp3")
self.brick_sound = pygame.mixer.Sound("sounds/brick.mp3")
self.lose_sound = pygame.mixer.Sound("sounds/lose.mp3")
```

Step 10 — Play the lose sound

Now we'll connect it to the **life-loss** event.

In `game/game_engine.py`, find this section in `update()` :

```
</> Python   Run  
if self.ball.y - self.ball.radius > self.height:  
    self.lives -= 1
```

Immediately after:

```
</> Python   Run  
self.lives -= 1
```

add:

```
</> Python   Run  
self.lose_sound.play()
```

Step 11 — Add a win sound

Get a short sound that feels like a **success / victory / level complete** sound.

Place it in:

```
sounds/  
├─ paddle.mp3  
├─ brick.mp3  
├─ lose.mp3  
└─ win.mp3 
```

Step 12 — Load `win.mp3`

In `game/game_engine.py`, you currently have:

```
</> Python   Run  
self.paddle_sound = pygame.mixer.Sound("sounds/paddle.mp3")  
self.brick_sound = pygame.mixer.Sound("sounds/brick.mp3")  
self.lose_sound = pygame.mixer.Sound("sounds/lose.mp3")
```

Add one line:

```
</> Python   Run  
self.win_sound = pygame.mixer.Sound("sounds/win.mp3")
```

So the complete section is:

```
</> Python   Run  
self.paddle_sound = pygame.mixer.Sound("sounds/paddle.mp3")  
self.brick_sound = pygame.mixer.Sound("sounds/brick.mp3")  
self.lose_sound = pygame.mixer.Sound("sounds/lose.mp3")  
self.win_sound = pygame.mixer.Sound("sounds/win.mp3")
```

Step 13 — Play the win sound

Now we'll connect it to the existing win condition.

In `game/game_engine.py`, find this part of `update()` :

 Python



 Run

```
if all(not b.alive for b in self.bricks):  
    self.game_over = True  
    self.result = "win"
```

Add:

 Python



 Run

```
self.win_sound.play()
```

So it becomes:

 Python



 Run

```
if all(not b.alive for b in self.bricks):  
    self.game_over = True  
    self.result = "win"  
    self.win_sound.play()
```

